

Processzusok kommunikációja, versenyhelyzetek, kölcsönös kizárás. Konkurens és kooperatív processzusok. Kritikus szekciók és megvalósítási módszereik: kölcsönös kizárás tevékeny várakozással (megszakítások tiltása, változók zárolása, szigorú váltogatás, Peterson megoldása, TSL utasítás). Altatás és ébresztés: termelő-fogyasztó probléma, szemaforok, mutex-ek, monitorok, Üzenet, adás, vétel. Írók és olvasók problémája. Sorompók.

Alapok

- a processzusokat a rendszerben futó folyamatok
- futó processzusok típusai
 - **Független processzusok**
 - olyan folyamatok amelyek nincsenek hatással más folyamatok futására és őket sem befolyásolják más folyamatok
 - saját adatterületen dolgoznak, nincs közös memóriájuk vagy változójuk
 - **Konkurens (látszólag párhuzamos) processzusok**
 - egy CPU esetén folyamatok nem egyszerre futnak, hanem az operációs rendszer gyorsan váltogat közöttük (időosztás), ami a felhasználó számára úgy tűnik mintha párhuzamosan futnának (látszatpárhuzamosság, virtuális CPU koncepció, mintha minden folyamatnak lenne külön CPU-ja)
 - **Kooperatív (együttműködő) processzusok**
 - olyan folyamatok, amelyek egymással szoros kapcsolatban vannak
 - közös feladaton dolgoznak, ehhez valamilyen közös tárolóterületet (pl: osztott memória, közös fájl) használnak
 - elengedhetetlen ezeknél a szinkronizáció és a kommunikáció
 - nehezebb leprogramozni, de lehetővé teszi:
 - a moduláris felépítést (egy komplex feladatot több kis processzus hajt végre), ha több CPU van nő a sebesség
 - az adatok megosztása több kliens között (pl: adatbázis-szerver)

InterProcess Communication (IPC)

- az IPC azon módszerek összessége, melyek lehetővé teszik a folyamatok közötti információcserét, kommunikációt
- 3 fő témakört kell megoldania:
 - **információátadás**
 - hogyan juttat el adatot egyik folyamat a másik folyamathoz?
 - **kölcsönös kizárás**
 - hogyan biztosítható, hogy két vagy több folyamat ne zavarja egymást kritikus tevékenységek közben?
 - probléma: a versenyhelyzet, megoldás rá: kölcsönös kizárás
 - **sorrendiség (függőségek)**
 - hogyan tartható fent a helyes sorrend, ha az egyik folyamat egy másik eredményére vár?

Versenyhelyzet (Race Condition)

- ha több folyamat is osztozik valamilyen közös erőforráson (pl: memória vagy fájl), akkor elkerülhetetlenül fellépnek időzítési problémák, ez a versenyhelyzet
 - ha két vagy több folyamat is egyszerre olvas vagy ír közös adatot és a végeredmény attól függ, hogy pontosan milyen sorrendben futottak le az utasításai
- példa (háttérnyomtatás)
 - a nyomtatandó fájlok neveit a folyamatok egy közös könyvtárba írják
 - két fontos változó van:
 - out (következő nyomtatandó fájl)
 - in (következő szabad hely a listában)
 - hiba folyamata:
 1. "A" processzus kiolvassa az in értékét (pl: 7) és eltárolja egy saját változóba
 2. ekkor kontextus-csere történik "B" folyamat kapja meg a CPU-t
 3. "B" is kiolvassa az in értékét (még mindig 7) és beírja a saját fájlnevét a lista 7. helyére (in érték), majd növelő in értékét 1-el, azaz 8-ra
 4. mivel befejeződött a "B" folyamat, ezért "A" visszakapja a CPU-t és a korábban a saját lokális változóban lévő 7-es értéket felhasználva a lista 7-es helyén lévő beírt fájlnevet felülírja
 5. a felülírás miatt a "B" bejegyzése sose kerül kinyomtatásra

Kritikus szekció

- a versenyhelyzetek elkerülése érdekében a programok futását kontrollálni kell
- a programnak azt a részét, kódrészletét nevezzük **kritikus szekciónak**, amelyben a közös erőforráshoz fér hozzá a program
 - osztott erőforrás lehet közös memória, változó vagy akár fájl is
- **cél megakadályozni, hogy egyszerre több folyamat is tartózkodjon a saját kritikus szekciójában, ha azok ugyanazt az osztott erőforrást használják**
 - szinkronizációs mechanizmusok (szemaforok, mutexek és monitorok)
- a megvalósítása fő szempont operációs rendszer tervezésekor

Kölcsönös kizárás

- olyan technika, mely biztosítja, hogy ha egy processzus épp a kritikus szekciójában van és osztott erőforrást használ akkor a többi processzus számára tilos legyen ebben a szekcióba belépni (vagy ugyanazt az erőforrást használni)
- OS tervezéskor alapszempont
- azért hogy jó legyen egy algoritmus mely kölcsönös kizárást valósít meg, 4 feltételnek kell megfelelnie egyidejűleg
 - **kizárólagosság (kölcsönös kizárás)**
 - egyszerre legfeljebb egy folyamat tartózkodhat a saját kritikus szekciójában
 - **függetlenség a hardvertől**
 - semmilyen feltételezés nem tehető fel a processzorok sebességéről és számáról sem (pl: a megoldásnak működnie kell akkor is ha egyik CPU sokkal gyorsabb mint a másik)

- **kritikus szekción kívüli blokkolás tilalma**
 - egy kritikus szekción kívül futó folyamat nem blokkolhat másokat, akik be akarnak lépni
 - ha egy folyamat lefagy a nem kritikus részén, akkor ez nem akadályozhatja meg a többi abba, hogy belépjenek a saját kritikus szekciójukba
- **véges várakozás (éhezésmentesség)**
 - egyetlen folyamatnak sem szabad végtelen ideig várnia arra, hogy beléphessen a kritikus szekciójába

Kölcsönös kizárás megvalósítása Tevékeny várakozással

- **tevékeny várakozás (spinlock):** a várakozó processzus egy ciklusban folyamatosan egy változó értékét teszteli, amíg belépési engedélyt nem kap
- hátránya a CPU pazarlás és a prioritás-inverzió veszélye
 - **prioritás-inverzió:** lényege, ha van egy M magas, egy K közepes és egy A alacsony prioritású folyamat és ha a magas és alacsony prioritású közös erőforrást használ és az alacsony jön először, majd a közepes elveszi tőle a CPU-t és blokkolódik akkor hiba jön be magas prioritással a folyamat várnia kell az erőforrásra
- CPU idő pazarlás miatt csak akkor használjuk a tevékeny várakozást ha a várakozási idő rövid lesz

1. Megszakítások tiltása

- a processzus közvetlenül a kritikus szekcióba lépés után letiltja az összes megszakítást, majd a kilépés előtt visszakapcsolja azokat
 - azért megoldás mert ha nincs megszakítás, akkor az ütemező sem tudja elvenni a CPU-t a folyamattól (nincs kontextus-csere)
- ez egy hardveres megoldás
- problémái
 - veszélyes, mert ha egy felhasználói program elfelejti visszakapcsolni a megszakításokat, a rendszer leállhat
 - több processzoros rendszerekben hatástalan, mivel a tiltás csak arra a CPU-ra vonatkozik, amelyik kiadta
- csak kernelen belül, rövid időre (pl: adminisztratív műveletek)

2. Záróváltozók

- egyetlen közös szoftveres változót (lock) használunk, kezdeti értéke 0
- ezt a változót belépés előtt teszteljük ha 0, átállítjuk 1-re és belépünk, ha 1, akkor várunk
- problémája, hogy versenyhelyzet alakulhat ki és két folyamat egyszerre láthatja 0-nak a változót és mindkettő beléphet, mielőtt a másik 1-re állítaná

3. Szigorú váltogatás

- folyamatosan tesztel és egy turn változó jelzi ki következik, a folyamatok egymást követik a kritikus szekcióba, ezzel elérjük, hogy egyidejűleg csak 1 legyen a kritikus szekcióba
 - ha $turn = 0$, akkor a 0-s processzus, stb
 - $while (turn \neq 0);$ (várakozás) $\rightarrow critical_region(); \rightarrow turn = 1;$
- biztosítja a kölcsönös kizárást, de megsérti a 3. feltétel (lasabb folyamat feltarthatja a gyorsabbat a kritikus szekción kívül is)

4. Peterson megoldás

- ez az első helyes szoftveres megoldás (1981), mely **két processzus esetén garantálja a kölcsönös kizárást** és nem igényel hardveres támogatást
- két változót kombinál
 - egy **sorrend jelző változót** (turn), ez mondja meg kinek van elsőbbsége
 - egy **szándékjelző tömböt** (interested tömb)
 - ebben minden folyamatnak van egy saját bitje (igaz/hamis)
 - ha a folyamat be akar lépni a kritikus szekciójába akkor a saját értékét igazra állítja
- belépés: $interested[i] = TRUE; turn = i;$
 - mikor belép a kritikus szekciójába akkor jelzi a szándékát és saját szándékjelzőjét igazra állítja
 - majd a turn változót beállítja saját azonosítójára (udvariassági lépés)
 - ezzel maga elé engedi a másik folyamatot
- várakozás: $while(interested[j] \wedge turn == i);$
 - belépés után jön a várakozás (tevékeny várakozás), egy ciklusban a folyamat addig vár, míg a két feltétel egyszerre nem teljesül
 - a turn változó az ő azonosítóját mutatja **ÉS** a másik processzus is jelzi a belépési szándékát
 - ha a feltétel hamis akkor azonnal belép várakozás nélkül
- befejezés: $interested[i] = FALSE$
 - mikor a processzus befejezi a munkáját hamisra állítja saját szándékjelzőjét, ezzel utat enged a másiknak aki várakozik
- tehát röviden **akkor léphetünk be, ha a másik nem akar, VAGY ha a másik udvarisan átadta a kört**
- megoldás, mert ha egyszerre is akar 2 folyamat belépni akkor a turn változóba a saját azonosítójukat írják, és a turn egyszerű változó így az utolsó írás fog érvényesülni, ezért aki utoljára irt bele az fog várakozni, mert a turn az ő azonosítóját fogja tartalmazni

5. TSL utasítás (Test and Set Lock)

- TSL olyan gépi, hardver szintű utasítás, melyet a modern processzorok biztosítanak a szinkronizáció megkönnyítésére
- lényege az atomiság, azaz a utasítás két lépést hajt végre úgy, hogy közben semmi más, még egy másik processzor sem férhet hozzá a memóriához
 - az utasítás kap egy memóriahelyet (LOCK változónak hívjuk) és egy regisztert (RX)
 - majd egyetlen megszakíthatatlan, oszthatatlan műveletbe a következők történnek:
 - atomi módon olvassa be a LOCK változó aktuális értékét és beleteszi az RX regiszterbe
 - majd a LOCK változó helyére ír a memóriába egy nem nulla (általában 1) értéket
 - a művelet során a CPU zárolja memóriasínt és a memória elérése más CPU-knak tilos lesz
 - miután az utasítás lefutott, a program megvizsgálja a regiszterbe került (régi) értéket
 - ha 0 volt, akkor a kapu nyitva volt a processzus sikeresen bezárta, mert TSL 1-et írt oda és beléphet a kritikus szekcióba
 - ha 1 volt a régi érték akkor valaki más már korábban lezárta, ilyenkor a folyamat visszaugrik az utasítás elejére és újra megpróbálja (tevékeny várakozás / spinlock)
 - kilépéskor:
 - amikor a folyamat végzett, egy egyszerű tároló utasítással visszaírja a 0 értéket a LOCK változóba

Kölcsönös kizárás megvalósítása: Magasabb szintű szinkronizáció (altatás és ébresztés)

- a tevékeny várakozás legnagyobb hibája, hogy pazarolja a CPU-időt és prioritás-inverziót idéz elő
- olyan megoldás kell ami blokkolja a folyamatot, amíg az nem tud tovább lépni
 - **sleep()**
 - rendszerhívás, mely felfüggeszti a hívó processzust (blokkolt állapotba teszi), amíg egy másik processzus fel nem ébreszti
 - **wakeup(process)**
 - felébreszti a megadott folyamatot (futáskész állapotba teszi)
- a klasszikus példa a szinkronizáció hibáira a **Termelő-Fogyasztó** probléma
 - van egy fix méretű tárhely (puffer), ide a **termelő** adatokat tesz be, míg a **fogyasztó** pedig kiveszi azokat

- ha a tároló tele van:
 - akkor a termelő elalszik, ha elemet akar berakni a tárolóba
 - felébreszti a fogyasztó őt, ha egy elemet a fogyasztó kivesz a tárolóból
- ha a tároló üres:
 - a fogyasztó elalszik, ha ki akar egy elemet venni
 - felébreszti a termelő őt, ha legyártott egy elemet
- probléma: a termelő nem tehet beletöbbet, ha a puffer tele van
- elveszett ébresztés (lost wakeup) jelenség is előfordulhat:
 - a puffer üres, a fogyasztó kiolvassa a count értéket: 0
 - ekkor az ütemező megszakítja a fogyasztó folyamatot (még mielőtt elaludna) és a termelő folyamat kapja a CPU-t
 - a termelő folyamat betesz egy elemet a pufferbe, a count 1 lesz és innen látja, hogy eddig 0 volt, tehát azt hiszi a fogyasztó folyamat alszik, ezért felébreszti (wakeup(fogyasztó))
 - mivel a fogyasztó technikailag nem aludt ezért az ébresztés “elveszik”
 - a fogyasztó visszakapja a CPU-t és végrehajtja a sleep()-et, mert ő korábban 0-t olvasott ki
 - a termelő előbb-utóbb megtölti a puffer-t, majd elalszik, az eredmény: **holtpont (deadlock)**

Kölcsönös kizárás megvalósítása: Szemaforok

- Dijkstra javasolta 1965-ben
- egy szemafor egy nem negatív egész változó **melyben számolhatjuk az ébresztéseket** a későbbi felhasználás céljából
 - használjuk **kölcsönös kizárásra** (értéke 1) és **szinkronizációra** (sorrendiségre) (pl: tele/üres helyek számlálásra) is
- 2 atomi (megszakíthatatlan, azaz addig még nem fejeződnek be addig a szemafort más processzus nem éri el) művelete van:
 - **down (vagy P, Wait)**
 - csökkenti az értékét
 - ha érték 0-nál nagyobb volt akkor a folyamat továbbhalad
 - ha 0 volt az érték akkor a folyamat alvó állapotba kerül, azaz blokkolódik anélkül, hogy befejezné a műveletet
 - **up (vagy V, Signal)**
 - növeli a szemafor értékét
 - ha volt alvó folyamat ezen a szemaforon akkor egyet felébreszt, ami befejezheti a saját down műveletét
- mikor használjuk?
 - készletkezelés, például egy erőforrásból több is van
 - jelzés (signaling), ha az egyik folyamatnak szólnia kell a másiknak mikor kész van, hogy a másik folytassa

- megoldja a termelő-fogyasztó hibáját, mert emlékszik a jelekre
 - ha a termelő küld egy up-ot, az akkor is növeli a szemafor értékét, ha senki se alszik rajta, így a fogyasztó később látni fogja, hogy van dolga
- példa
 - Egy könyvtár, ahol egy könyvből 5 példány van, itt a szemafor értéke kezdetben 5
 - mindenki, aki kivesz egyet, végrehajt egy down műveletet (az érték csökken), ha elfogyott (érték = 0), a következő olvasónak várnia kell a könyvtárban (elalszik)
 - amikor valaki visszahozza, up műveletet végez, és az érték nő, a várakozó pedig megkapja a könyvet

Kölcsönös kizárás megvalósítása: Mutex-ek (bináris szemafor)

- szemafor egy speciális, egyszerűsített változata, amit csak kölcsönös kizárásra használunk
 - pl akkor használjuk ha van egy darab közös erőforrás és biztosítani akarjuk, hogy egyszerre csak egy folyamat férjen hozzá
- értéke csak bináris lehet (2 állapota)
 - 0 (nyitott, szabad)
 - 1 (zárt, foglalt)
- használata:
 - mutex_lock
 - ha a mutex szabad (értéke 0), lezárja és a folyamat belép a kritikus szekcióba
 - ha pedig a mutex foglalt (értéke 1) akkor a hívó blokkolódik
 - mutex_unlock
 - feloldja a zárat és ha van várakozó, felébreszti
 - **a mutexnek van gazdája csak az oldhatja fel aki lezárta**
- példa
 - van egy mosdó, ennek a kulcsa a mutex
 - aki be akar menni, elkéri a kulcsot (lock), ha a kulcs ott van, beviszi, magára zárja az ajtót
 - aki utána jön, látja, hogy nincs kulcs, ezért leül és vár (elalszik)
 - amikor az első kijön, visszateszi a kulcsot (unlock), és szól a várakozónak, hogy mehet
- TSL vs Mutex
 - TSL tevékenyen várakozik, ezzel égeti a CPU-t, míg a Mutex viszont elaltatja a folyamatot, ezzel a CPU-t átadja másik folyamatnak

Kölcsönös kizárás megvalósítása: Monitorok

- a szemaforok használata nehéz, könnyen el lehet rontani a sorrendet, ami holdponthoz vezet, helyett a **monitor egy magasszintű programozási nyelv konstrukció** (JAVA-ban a synchronized)
- a monitor egy modul, osztály, mely adatokat és eljárásokat tartalmaz
 - az eljárásai automatikusan kölcsönös kizárással futnak, azaz egyszerre csak egy processzus tartózkodhat aktívan a monitoron belül, ha egy másik is hívná valamely eljárását a monitornak akkor várakoznia kell

- **feltételi változókkal** a szinkronizációt segítik, pontosabban azt, hogy monitoron belül is lehessen várni (pl: üres buffer esetén) 2 művelet van:
 - wait(c)
 - a folyamat elalszik a *c* feltételen és ideiglenesen elhagyja a monitor, hogy más bejöhessen
 - signal(c)
 - felébreszti az egyik folyamatot, mely a *c* feltételre vár

Kölcsönös kizárás megvalósítása: Üzenetküldés

- az üzenetküldés az egyetlen mód a kölcsönös kizárásra elosztott rendszerekben, **ahol nincs közös memória**, mert a folyamatok különálló, hálózaton összekapcsolt gépeken futnak, amúgy is lehet, de nem éri meg mert lassabb
- működése
 - a folyamatok struktúrált adatsomagokat, üzeneteket küldenek az operációs rendszeren keresztül
 - hasonlít a szemaforokhoz, de az üzenetek nem csak jeleznek, hanem adatokat is szállítanak
- itt a programozónak explicit módon hívnia kell a küldő/fogadó műveleteket, a primitíveket vagy rendszerhívásokat:
 - send(destination, message)
 - üzenet küldése egy célállomásnak, mely lehet egy folyamat azonosítója, vagy akár egy köztes tároló is
 - receive(source, message)
 - üzenet fogadása, egy adott forrástól
 - a hívó általában blokkolódik, ameddig meg nem érkezik az üzenet
- problémái:
 - elveszett üzenetek
 - a hálózaton az üzenet elveszthet
 - megoldás rá a nyugtázás, azaz ha a fogadó egy **nyugtázó üzenetet (ACK)** küld vissza és ha a küldő egy bizonyos időkorláton (timeout) belül nem kapja ezt meg akkor újraküldi
 - hitelesítés
 - biztonsági szempont, azaz biztosnak kell lenni benne, hogy a megfelelő folyamat kapja meg az üzenetet, illetve, hogy a feladó is az legyen akinek mondja magát
 - teljesítmény
 - üzenetkezelés általában lassabb mint a többi IPC (szemafor, mutex) mert adatok másolásával és hálózati késletetéssel jár
- azonosítás / címzés
 - lehet **közvetlen**, mikor folyamat ID-re címezzük
 - lehet **közvetett**, mikor például egy postaládán keresztül küldjük

- **Közvetett kommunikáció (levelesláda)**

- ahelyett, hogy közvetlenül folyamatoknak küldenénk az adatot, inkább egy levelesláda adatszerkezetet használunk, mely adott számú üzenet tárolására alkalmas
 - küldő a levelesládába teszi az üzenetet, a fogadó pedig onnan veszi ki
 - mikor a levelesláda tele van akkor a küldő blokkolódik, ha pedig üres akkor a fogadó fog blokkolódni

- **Közvetlen kommunikáció (randevú)**

- ha elhagyjuk az ideiglenes tárolást (postaláda) akkor a küldő és fogadó közvetlenül találkozik
 - mint egy fizikai találkozó, ha a küldő folyamat ér oda előbb az üzenetével, meg kell várja a fogadó folyamatot, ha pedig a fogadó ér oda előbb akkor meg kell várni a küldő folyamatot mely hozza az üzenetet
 - amikor mindkettő folyamat odaér akkor az adatátadás megtörténik és mindkét folyamat megy tovább
- előnye a könnyű megvalósítás, mivel nincs szükség memória kezelésre
- hátránya, hogy kevésbé rugalmas, szorosabban kapcsolódnak egymáshoz a folyamatok

Klasszikus IPC problémák és speciális eszközök

- **Írók és olvasók problémája**

- a probléma egy közös adatbázis, vagy fájl elérését modellezi, ahol két fajta folyamat van:
 - **olvasó folyamatok**
 - csak adatokat kérdeznek le, nem módosítanak semmit
 - **író folyamatok**
 - csak módosítják az adatokat
- szinkronizációs szabályok:
 - több olvasó egyszerre használhatja az adatbázist
 - mivel nem zavarják egymást
 - ha egy író dolgozik senki más nem férhet hozzá az adatokhoz
 - se író, se olvasó
 - ha egy olvasó bent van, akkor újabb olvasók bejöhetnek, de az író folyamatoknak várniuk kell
- megvalósítás szemaforokkal
 - van egy számláló az olvasókra (read_count), egy mutex a számláló védelmére és egy szemafor (db) az adatbázis eléréséhez
 - menete
 - az első olvasó, aki érkezik az végrehajt egy down(db) mutex műveletet, amivel lezárja az írók előtt a db-t
 - az utolsó olvasó, aki távozik az végrehajt egy up(db) műveletet, mellyel felszabadítja a db-t az íróknak

- éhezés
 - ha folyamatosan jönnek az olvasó folyamatok akkor az író folyamatok várnak és várnak, éheznek
 - megoldás, hogy ha egy író érkezik akkor az utána érkező olvasókat már nem engedik be, amíg az író sorba nem került
- **Sorompók**
 - a sorompó egy olyan szinkronizációs pont, melyet általában a folyamatok egy csoportjának összehangolására, használnak
 - főként fázisokra osztott párhuzamos algoritmusoknál
 - **nem egy erőforrást véd, hanem a folyamatok egy csoportját fogja össze**
 - működése
 - a folyamatok a saját feladatukkal elindulnak
 - mikor végez a folyamat a fáziséval, eléri a barriert, avagy a sorompót és ott megáll, azaz blokkolódik
 - a sorompónál a folyamatok addig várnak még az összes résztvevő folyamat meg nem érkezik
 - amint az utolsó is megjött, az összes folyamat átléphet a sorompón, azaz folytathatja a következő fázist
 - főleg párhuzamos algoritmusoknál, ahol a munka fázisokra van osztva használjuk
 - senki nem kezdheti el a következő fázist, míg a jelenlegi fázist a csoporthoz tartozó összes folyamat el nem végezte
 - pl: Olyan ez, mint egy csapatmunka a kiránduláson. Megbeszéljük, hogy mindenki a saját tempójában sétál fel a hegytetőre, de senki nem indul el lefelé a másik oldalra, amíg az utolsó ember is meg nem érkezett a csúcsra

Összefoglaló

- a folyamatok biztonságos együttműködése alapvető az operációs rendszerekben
 - a versenyhelyzetek elkerülésére a kölcsönös kizárást alkalmazzuk
- míg a tevékeny várakozás (TSL, Peterson, stb) pazarolja az erőforrásokat, addig az alvó/ébresztő mechanizmusok (szemaforok, monitorok) hatékonyabbak
 - elosztott rendszerekben pedig az üzenetküldés veszi át a közös változók szerepét
- nincs legjobb kommunikációs mód, mert problémától függ
 - rövid várakozás esetén a TSL / spinlock jó
 - hosszú várakozás esetén pedig a szemafor vagy mutex a jó